Server update runbook — SkillSpan on alwaysdata

Everything needed to go from "SSH" to "updated and verified". Copy the blocks as they are; nothing here needs editing per deploy.

---

1. Connect

```
ssh skillspan@ssh-skillspan.alwaysdata.net
```

- host: `ssh-skillspan.alwaysdata.net`, port `22`
- main user: `skillspan`
- second user (backend colleague): `skillspan_qasem`
- browser terminal (no SSH client): `https://ssh-skillspan.alwaysdata.net`

First connection asks about the host key — answer `yes`. The fingerprint is published in **Remote access > SSH/SFTP** in the alwaysdata panel; compare it before answering if you want to be strict.

---

2. Update — the whole thing

```
cd ~/www/skillspan
cp storage/api-docs/api-docs.json ~/api-docs.json.bak 2>/dev/null
git checkout -- storage/api-docs/api-docs.json
git pull
php artisan migrate --force
php artisan db:seed --class=MvpCatalogSeeder --force
php artisan optimize:clear
php artisan config:cache
```

Run it line by line the first time so you can see which step complains. Once it is routine, the whole thing can go on one line joined with `&&`.

What each line is for

line	why
`cd ~/www/skillspan`	the app root; the web root is `public/` inside it
`cp … ~/api-docs.json.bak`	backup of a **generated** Swagger file that the host rewrites
`git checkout -- storage/api-docs/api-docs.json`	without this the pull aborts: the tracked file is dirty on the host
`git pull`	the branch is `feature/authentication`, not `main`
`php artisan migrate --force`	`--force` because `APP_ENV=production`; without it Laravel prompts and the deploy stalls
`php artisan db:seed --class=MvpCatalogSeeder --force`	applies the MVP catalogue policy (which roles/specializations are offered)
`php artisan optimize:clear`	**not optional** — a stale routes cache on this host serves 404/405 for routes that exist
`php artisan config:cache`	the host runs from cached config

---

3. Check that it actually landed

From any machine with `curl`:

```
curl -s -o /dev/null -w "up=%{http_code}\n" https://skillspan.alwaysdata.net/up
curl -s -o /dev/null -w "roles=%{http_code}\n" -H "Accept: application/json"
https://skillspan.alwaysdata.net/api/v1/career-roles
```

- `up=200` — the app is alive
- `roles=401` — the route exists and is protected. **404 means the code is not deployed**, no matter what `git log` says.

What each kind of change looks like from outside:

change	marker
new API route	`404` → `401` (bearer) or `302` (session panel)
new validation rule	a request that **must** fail returns `422` with the new `errors` key
new OpenAPI field	the string appears on `https://skillspan.alwaysdata.net/docs`
data / seeding change	the numbers in the API response move (e.g. career roles 146 → 93)

`404` from a route that `php artisan route:list` shows is the routes cache — run `php artisan optimize:clear` again.

---

4. When it breaks

`git pull` **aborts on** `storage/api-docs/api-docs.json` Already handled by the two lines above. If it still aborts, the backup is at `~/api-docs.json.bak`.

`Permission denied` **on** `storage/` **or** `bootstrap/cache/` A second SSH user wrote files there and now owns them.

```
chmod -R g+w storage bootstrap/cache
```

If that is not enough, the files are owned by the other user and cannot be re-owned without root. From then on, let **one** person run deploys.

`Class "…" not found` **right after a pull** Only happens when `composer.json` / `composer.lock` changed:

```
composer install --no-dev --optimize-autoloader
```

`unable to update local ref` **on Windows (local machine, not the server)** A stale lock from an interrupted pull:

```
rm -f .git/refs/remotes/origin/feature/authentication.lock
git fetch origin
```

Site is up but a page 500s after a pull Read `storage/logs/laravel.log` on the server. The usual cause is a migration that was not run.

---

5. Never on this server

- `php artisan migrate:fresh` / `db:wipe` — the database is **shared and live**.
- `php artisan serve`, queues, workers, any background process — alwaysdata suspends accounts for it.
- `git reset --hard` — the working tree is the deployed site.
- `git checkout -- .` — it discards every local change on the host, not just the file you meant.

---

6. Rolling back a bad deploy

```
cd ~/www/skillspan
git log --oneline -5
git checkout <previous-commit>
php artisan optimize:clear
php artisan config:cache
```

Migrations that already ran are **not** undone by this, and normally they do not need to be: the ones in this project are additive (a new column with a default). Rolling the schema back needs `php artisan migrate:rollback --step=1`, which is a deliberate decision, not a reflex.

---

7. Things worth knowing before you deploy

- The deploy ships the **branch**, so pushing to `main` does nothing here.
- `.env` is git-ignored: `git pull` never refreshes it. If a new key is added, edit it by hand and run `php artisan config:cache` afterwards.
- Cached config means **any dashboard/env change needs a redeploy** to take effect.
- The frontend is a separate repo on Vercel and deploys itself on push to `main`. A backend deploy does not touch it, and vice versa.